An Intelligent Methodology to Enhance Requirements Engineering in Multidisciplinary Projects

Ali Salmani[1], Alireza Imani[1], Majid Bahrehvar[1], Linda Duffett-Leger[2], Mohammad Moshirpour[1]

[1]Department of Electrical and Software Engineering, Schulich School of Engineering, University of Calgary, Calgary, Canada
[2]Faculty of Nursing, University of Calgary, Calgary, Canada

{ali.salmani1, alireza.imani, majid.bahrehvar, mmoshirpour, linda.duffettleger}@ucalgary.ca

2022 IEEE Canadian Conference on Electrical and Computer Engineering (CCECE) | 978-1-6654-8432-9/22/$31.00 ©2022 IEEE | DOI: 10.1109/CCECE49351.2022.9918286

The multidisciplinary nature of a team has been identified as one of the success criteria of both user-centric approaches and agile methods. Stakeholder involvement is considered to be essential for agile processes in order to meet project objectives and ensure results are aligned with stakeholder expectations. However, establishing a collaborative process involving designers, programmers, stakeholders, and users can be challenging; particularly during the requirements engineering stage. Agile methodologies, such as scrum, offer a powerful way of effectively managing software projects and generates a great deal of useful data through tools such as Jira and GitHub. The aim of this research is two folds: 1) analyze the project data from aforementioned sources using process mining techniques to discover deficiencies in the software development process, and 2) propose an automated effort estimation process to address the identified challenges in this study and provide decision support in the development process. This approach is applied to a case study of a virtual healthcare intervention system. The results are indicative that these enhancements helped boost the decision-making and release planning processes by providing the development team a more clear picture, which ultimately mitigated the number of change requests.

***Index Terms*—Multidisciplinary, Agile methodology, Requirements Engineering, Process Mining, Effort Estimation**

I. Introduction

Scrum is an agile methodology that is widely used to address adaptive problems, delivering products quickly and effectively, and deliver value to clients [1]. When it comes to software companies, choosing the right software process is one of the key challenges. Selecting software a software process is usually based on previous project success [2]. However, scrum in software development is analogous to the rules of the sport. Knowing the rules alone will not suffice. It requires a deep understanding of modern engineering practices in order to be successful. A team's effectiveness is dependent on its members' interpersonal and intrapersonal intelligence and maturity. Consistent and continuous coaching is needed throughout the process [3]. In Scrum, requirements engineering is an informally specified process that depends heavily on individual skill and knowledge [4]. Requirements engineering happens iteratively during the development process, not during a closed phase at the beginning [4], [5].

This paper presents a real-world case study: a virtual health intervention system developed by a multidisciplinary team that includes nurses and software engineers. To measure the effectiveness of our approach, we will use some data that has been collected to perform some process mining on the software development process. By leveraging data from sources like GitHub and Jira, process mining can provide insight into how software development is progressing [6]. By analyzing the results of process mining, it is possible to diagnose and improve the software development process and ultimately enhance the end product. It has been determined from our study that most challenges were caused by poor communication and lack of domain knowledge throughout the process, which can be attributed to the team's multidisciplinary nature. After analyzing the experience gained during the project and reviewing the literature, an automated solution has been developed to address the identified challenges.

To overcome the challenges encountered during software development, a decision support tool has been developed. This tool provides domain experts with an estimation of the amount of effort each solution will require. This tool would allow domain experts to select the best solution based on process data and intelligent decision support, aiding the release planning of the software and reducing the number of requirement changes during development.

The remainder of the paper is organized as follows. Section II discusses the background and related work in this area. The purpose of section III is to provide an overview of the multidisciplinary project that has been examined and evaluated as our case study. As part of this section, we will also provide an overview of the software development approach we have adopted. Section IV summarizes how effective and productive our approach was in our multidisciplinary team. As part of section V, we will discuss the deficiencies of our approach and explain our intelligent decision support system. We will then demonstrate how such a tool can be used to reflect the results of our approach. In section VI, we will discuss our research conclusions and our future research plans.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:21:19 UTC from IEEE Xplore. Restrictions apply.

II. Background and Related Work

A. Agile in a Multidisciplinary Team

The agile methodology differs from other methods in that it does not have clear-cut steps [7]. In place of specific processes or tools, it's a methodology that stresses a set of best practices that emphasize the importance of people and their relationships [8]. Scrum is the popular Agile methodology that is capable to help teams move and learn faster and is mostly adopted by software companies. Based on a study on scrum, 56% of teams used scrum methodology because it visualizes the tasks on a workflow and divides complicated tasks into user stories [9]. By considering the percentage of hybrid methodologies that have been adopted in teams, scrum is used by 83% of teams and organizational design culture is one of the biggest challenges for implementing scrum [9].

The length of sprints can be altered, new roles can be added to handle specific responsibilities throughout the project, and all processes can be customized [10]. There have been several studies on tailoring scrum to deliver higher quality in the least amount of time. Sanches et al, found that direct communication between different roles in a scrum team caused constant interruptions in the development process [11]. To meet this challenge a new role was added to fulfill the communications needs between teams involved in a distributed scenario. Rodriguez et al, [12], introduced an Agile-coach role to help students improve their use of Agile methodologies and reinforce nontechnical skills by adding a scrum-based training model. There has been another study conducted on the effectiveness of agile in the Video Game Development (VGD) process. Researchers examined the challenges faced by a multidisciplinary team in developing a game development project. This study found that VGD had more needs than what is being met by the current agile frameworks. Scrum and Kanban frameworks, as well as other Agile frameworks, should often be modified to account for the different pipelines and milestones in the stages of the VGD process [13].

B. Process Mining

Process mining involves extracting information about process behavior from event logs and it is considered a data-oriented discipline [14]. For organizations reliant on information systems to conduct their daily operations, process mining can be of great value [15]. By analyzing the run-time event data collected by those systems, a process mining analysis identifies the sequence of activities and the collaborations of users in a business process. There are many studies on the application of process mining in software development [16], [17]. Rubin et al. [18] explains how software configuration management system information can be used to construct explicit software development process models, which are often implicit and chaotic. Based on their findings, Disco and the ProM tool can be used to obtain process models from real-world software projects[18].

In [19], Arias, Marques and Rojas have conducted a systematic mapping study to obtain an overview of the state of the art on how process mining has been applied with agile methodology. They have also addressed the most commonly used techniques and tools such as Disco and ProM, similar to other studies [19].

Van der Aalst [14] describes three types of mining processes: discovery, conformance, and enhancement. Discovery involves analyzing logs (event records) in order to build a process model. A process model (ideal process) and an event log (real process) are compared during conformance checking analysis to determine whether they differ. Enhancing an existing process model based on actual (and used) information about the process is the objective of enhancement analysis.

C. Managing requirements changes

One of the inevitable phenomena in agile software development is the requirements changes. According to the survey given in [20], requirement changes can be categorized into the following 11 groups: defect fixing, missing requirements, functionality enhancement, product strategy, design improvement, scope reduction, redundant functionality, obsolete functionality, erroneous requirements, resolving conflicts, and clarifying requirements. We captured and discussed requirement changes in sprint review meetings and reported requirement changes as new user stories to the project's Jira board.

Several studies have been conducted to meet the Requirement Change Management (RCM) challenges [21]–[23]. In [21], Shahzadi and her team conducted a framework to reduce the existing RCM gaps such as categorization problems, storage of implemented/unimplemented requirements, and stakeholder notifications. However, the framework may not be applicable for all teams with different circumstances. For instance, they have incorporated a new phase after submitting a new change request in their framework called the requirement categorization to prioritize and measure the scope of the required changes since the changes are not the same in complexity and time cost [21]. Categorizing the change requests will not be easy and straightforward in multidisciplinary teams with several different points of view.

In an Agile development team, Mahmud et al. [24] suggest using the mind-mapping technique to elicit and process requirements as part of requirements elicitation. A case study was used to evaluate the impact. Their experiment showed: 1) When mind mapping is adopted, the overall quality of the product backlog is significantly higher with "less expert" product owners and at least not worse with more expert product owner. 2) Mind-mapping and any derived technique should be considered as a valuable good practice to be pursued for setting up a promising initial product backlog.

D. Task Difficulty Estimation

As part of a survey conducted by the Project Management Institute (PMI) in 2017, it was determined that although 69%

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:21:19 UTC from IEEE Xplore. Restrictions apply.

of software projects achieved their initial goals and business priority, 43% of these projects did not meet their budgets, 48% of them were late, and 32% of them failed due to budget shortages [25]. Based on [26], accurate cost and effort estimation is the key to preventing project cost overruns regardless of management competence and the financial strength of the contractor.

The software industry utilizes a number of different types of Software Effort Estimation (SEE) models. Many different types of effort estimation techniques have been developed over the past few decades, and a number of models have been developed for the purpose of achieving high effort estimation accuracy [27], [28]. A systematic review of SEE literature was conducted by [27] in order to identify the most widely used ML techniques. According to this study, the methods that were used for software effort estimation were Artificial Neural Networks, Fuzzy Logic, Genetic Algorithms, and Regression Trees. For effort estimations, most studies suggested using Line of Code (LoC) and Function Point (FP) software metrics.

In order to estimate the effort needed to develop a given software, regression analysis can be used; this is also known as algorithmic estimation. Regression-based estimation is performed by using FP and LOC as independent variables. In [29], COCOMO with KLOC is compared with COCOMO with FPA. It was found that the COCOMO model used with FPA provided more accurate results than the KLOC model used with Mean Magnitude of Relative Error (MMRE). Researchers studied the accuracy of the expert judgment and the composite model-based estimation of effort estimators in [30] . According to their findings, the two approaches have clearly distinct statistical differences and there is a substantial statistical difference between them. Consequently, they strongly agreed that model-based estimation generally results in more accurate effort estimates. Their final results indicate that the accuracy and effectiveness of their effort estimation techniques differ according to their goal of error measurement. Because of this, there is no method of estimating effort that would yield accurate results for all analysis goals.

A study involving data mining revealed that ensemble methods provide more accurate results than single-method approaches. It has inspired researchers to apply ensemble methods to various sub-fields of software engineering [31]. Ensemble effort estimation (EEE) is the idea that every technique has its own strengths and weaknesses. By integrating techniques through EEE, we can minimize these limitations, which may result in more accurate estimations. For ensemble-based methods, rules such as mean, median, Inverse Rank Weighted Mean-IRWM, etc. are combined. The effectiveness of ensemble effort estimation techniques has been evaluated in several empirical studies, with some focusing on homogeneous ensembles, heterogeneous ensembles, or both types [32]. To achieve high estimation accuracy, the ensemble-based methods must satisfy two conditions: high accuracy and heterogeneity. It should, therefore, be possible under the circumstances to use diverse and as accurate a base method as possible. Therefore, estimation errors caused by any base technique may be cancelled using any other base technique.

III. Case Study: Virtual Health Intervention Platform

A real-world virtual health intervention system, Robin's Nest, has been used to demonstrate our proposed AI-based approach in this paper. Robin's Nest is used as a vehicle to virtually offer several different healthcare and healthcare training programs. Some of these programs include VID-KIDS which offers medical support to new mothers, and ATTACH that provides virtual mental health support to caregivers in domestic abuse situations. Robin's Nest is developed as a multidisciplinary project involving multiple different classes of stakeholders including healthcare professionals, various administration roles, regulators, researchers, and patients. Robin's Nest provides an effective and efficient solution that leverages technology to tackle some of the contemporary challenges in healthcare including the COVID-19 pandemic, lack of resources, and long distances between medical professionals and their patients.This platform provides several convenient features such as scheduling, video conferencing, and data integration capabilities, as well as a comprehensive data analytics, visualisation and reporting dashboard for various user roles including the admins and researchers.

We have adopted scrum as our software development framework. In order to gather requirements, interdisciplinary meetings are scheduled with domain experts from various disciplines. The purpose of these meetings was to discuss both functional and non-functional requirements in detail. After gathering the requirements, Gherkin [1] was used to convert them to a semi-formalized text. We used behaviour-driven development (BDD) as part of our software development process; a collaborative approach that encourages collaboration among developers, quality assurance testers, and customer representatives. By establishing a shared understanding of how the application should behave using conversation and concrete examples, teams can formalize their understanding [33]. Thus, we collaborated with nursing stakeholders to document the requirements using Gherkin. Nursing staff also reviewed the documentation in order to ensure its validity and consistency.

The process evaluation is relied on the data that is gathered from Jira and GitHub. Jira provides reports that show the team efficiency and progress in given time intervals such as control charts, velocity, and burn-down charts. For instance, the burn-down chart shows the development team's progress in each sprint by showing the number of implemented user stories and story points. Furthermore, the process can be more scrutinized by analysing the data that is collected from GitHub and Jira. By considering the meta data such as the commit details for each task such as the creation date, the change status history of each task, and finding the relation between them, the overall flow of the software process can be reached.

[1] https://cucumber.io/

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:21:19 UTC from IEEE Xplore. Restrictions apply.

IV. Process Evaluation

Based on the reports provided by Jira, it can be concluded that most of the requirements were not acceptable as implemented. As a result, more details were examined in order to calculate the length of time it would take to fulfill the requirements. The term time-to-market [34] is used in industry to describe how long it takes from the conception of a product to its release on the market. Figure 1, illustrates some of the topics that have been discussed more frequently in the design sessions and have been changed in more detail. For instance, as part of the VID-KIDS application, nurses should be able to enter video calls into the video log and have it listed on the table. In order to fulfill this requirement, six user stories had to be created, and it took the product development team four sprints to complete them and reach the approval stage since the requirements were altered over time. This can be attributed to the ineffective communication between the multidisciplinary team, including during the requirements gathering meetings.

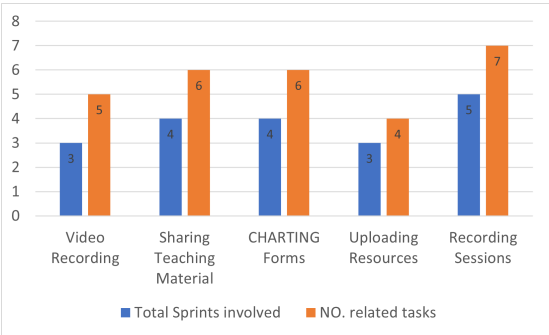

Fig. 1. Time taken for tasks that required several changes

Furthermore, in some cases, the software requirements changed during each sprint, which required the programmers to modify their previous work. When programmers spend more time on the requested changes for a specific user story, they will have less time for new user stories. Therefore, ineffective communication has resulted in a large amount of rework and has reduced the velocity of software development. Figure 2 represents the progress of the project in one sprint. During the sprint, 10 out of 16 tasks were left, which will be carried over to the next sprint.

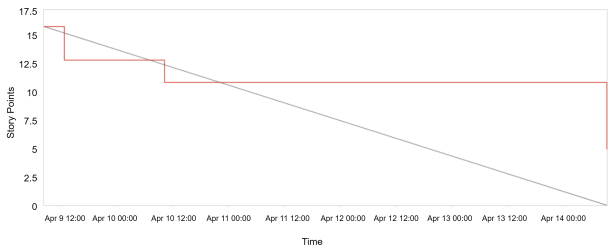

Fig. 2. The number of user stories remaining in a sprint

To calculate the rework rate, we used data automatically retrieved from Jira. The rework rate is defined as the ratio of rework hours to the productive labor hours. The rework hours is calculated by accumulating the number of hours dedicated on the requirements that have been implemented already. Figure 3 presents the monthly rework rates for five topics with the highest rework rate. The higher the value of the vertical axis, the higher the rework rate. For instance, the rework rate of the *Video Recording* requirement was 0.1 in August, 0.05 in September, 0.05 in October, and 0.2 overall in the following three months.

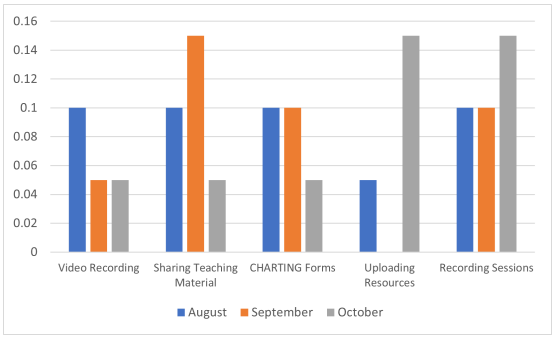

Fig. 3. Monthly Rework Rate

V. Discussions and Enhancements

A. Discussions

Based on the data and our analysis, it can be concluded that the following results are due to a set of inconsistencies between the software developers and the domain experts. As discussed earlier, there are a considerable number of tickets that have been created for a given requirement which shows that the requirement has been changed frequently. This requirement change might be due to a few reasons:

1) Plurality of Stakeholders

The number of nurses participating in the decision making sessions were many. It is important to note that participation in meetings was optional for some of the nurses. On the other hand, the agenda of such meetings included two main parts; first, decision making for the new tasks, and second, reviewing the features that were recently developed. Demonstrating the new features to the stakeholders that were not aware of the design decisions would often requirement change as they would communicate new ideas about the feature. While it is important to obtain the users' feedback about the features, the lack of central direction can cause inefficiencies in the development process.

2) Lack of Software Engineering Knowledge

Non-software engineering researchers often lack the technical prospective about how much effort is required to develop a feature. Furthermore, every requirement may have more than one solution to be considered. Consequently, it is hard for the non-technical stakeholders, such as the nursing researchers, to decide which solution would work depending on the timeline that they've considered for developing the application. Also,

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:21:19 UTC from IEEE Xplore. Restrictions apply.

Novice software developers may often lack this prospective, as it takes a vast amount of experience to develop.

B. Enhancement: Automating the Effort Estimation

In this paper, we've developed an intelligent approach to estimate the required effort for developing new features or implementing requirement changes. We seek to give the development teach including intelligent insights into how much effort is required for each tasks so that they can make effective decisions based on the software requirements. We have used BERT [35] as a Natural Language Processing (NLP) technique for clustering the tasks that already been created on Jira and GitHub according to their descriptions. The input of our model would be a short description of the requirements. The model lists the most similar tasks that have already been completed by making use of the cosine algorithm. The selection of the best similar task can be performed manually. Once the most similar ticket has been selected, the required effort will be estimated based on the required story points, the total time-to-market, etc..

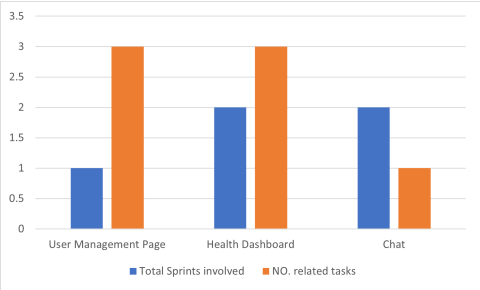

Fig. 4. Taken time for the tasks that required change requests

Based on the figure 4, there are 3 topics that were not resolved during the requirements gathering process as shown by the figure. The original development of these topics predates the use of our decision support tool. One example of such a topic is the Health Dashboard which took the development team less time than before once relying on our tool. The task was completed in three sprints and there were only two tasks related to this topic in the backlog, which represents a considerable improvement. On the other hand, The number of user stories completed in a sprint has increased in recent sprints, as indicated in figure 5; at the end of a sprint, there are only 20 user stories left to complete.

VI. Conclusion and Future Work

This paper highlights the challenges of a multidisciplinary team by leveraging process mining techniques to evaluate the team's software development process. The results showed several deficiencies which impacted the SDLC cause time loss including: high rework rates, a lack of mutual understanding between the team members, and occasionally changing requirements. Based on our findings after analysing the data obtained from Jira, one of the most important areas that

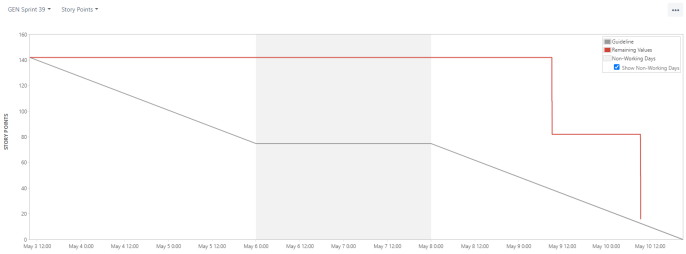

Fig. 5. The number of user stories remaining after effort estimation automation

could have been improved within our software development process was our approach to requirements engineering. Thus, our proposed solution is to implement an automated process for estimating the required effort of the new features based on the similar tasks that have already been done using BERT. The changes we made to our development process enable our team to make vast advances in productivity and progress by having access to intelligent decision support. In the future, we are going to adopt process mining tools such as ProM and Disco for better data collection and analysis. Furthermore, with respect to the results that have been discovered in this study, the investigation will concentrate on implementing the proposed solutions including creating an automated process for identifying the requirements and improving our effort estimation model.

References

[1] *Scrum - what it is, how it works, and why it's awesome*. [Online]. Available: https://www.atlassian.com/agile/scrum.

[2] A. M. M. Hamed and H. Abushama, "Popular agile approaches in software development: Review and analysis," *Proceedings - 2013 International Conference on Computer, Electrical and Electronics Engineering: 'Research Makes a Difference', ICCEEE 2013*, pp. 160–166, 2013. DOI: 10.1109/ICCEEE.2013.6633925.

[3] E. P. Wonohardjo, R. F. Sunaryo, Y. Sudiyono, N. Surantha, and Suharjito, "A systematic review of scrum in software development," *International Journal on Informatics Visualization*, vol. 3, no. 2, pp. 108–112, 2019, ISSN: 25499904. DOI: 10.30630/JOIV.3.2.167.

[4] A. Alhazmi and S. Huang, "Survey on Differences of Requirements Engineering for Traditional and Agile Development Processes," *Conference Proceedings - IEEE SOUTHEASTCON*, vol. 2020-January, 2020, ISSN: 07347502. DOI: 10.1109/SOUTHEASTCON44009.2020.9397492.

[5] E. M. Schön, J. Thomaschewski, and M. J. Escalona, "Agile Requirements Engineering: A systematic literature review," *Computer Standards & Interfaces*, vol. 49, pp. 79–91, Jan. 2017, ISSN: 0920-5489. DOI: 10.1016/J.CSI.2016.08.011.

[6] R. Marques, M. M. Da Silva, and D. R. Ferreira, "Assessing agile software development processes with process mining: A case study," *Proceeding - 2018 20th IEEE International Conference on Business Informatics, CBI 2018*, vol. 1, pp. 109–118, Aug. 2018. DOI: 10.1109/CBI.2018.00021.

[7] M. Jovanovic, A. Mesquida, N. Radaković, and A. Mas, "Agile retrospective games for different team development phases," *Journal of Universal Computer Science*, vol. 22, pp. 1489–1508, Apr. 2016. DOI: 10.3217/jucs-022-12-1489.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:21:19 UTC from IEEE Xplore. Restrictions apply.

[8] K. Beck, M. Beedle, A. Van Bennekum, *et al.*, "Manifesto for Agile Software Development Twelve Principles of Agile Software," 2001. [Online]. Available: http://www.agilemanifesto.org.

[9] *Latest Reports on the Most Popular Agile Methodologies — Adeva*. [Online]. Available: https://adevait.com/blog/remote-work/adopting-agile-the-latest-reports-about-the-popular-mindset.

[10] *Agile Scrum - Tailoring Scrum to Your Needs*. [Online]. Available: https://smartbear.com/blog/agile-scrum-tailoring-scrum-to-your-needs/.

[11] *A SCRUM-Based Process to Distributed Projects in Multidisciplinary Teams: A Case Study — IEEE Conference Publication — IEEE Xplore*. [Online]. Available: https://ieeexplore.ieee.org/document/8560649.

[12] G. Rodriguez, A. Soria, and M. Campo, "Measuring the Impact of Agile Coaching on Students' Performance," *IEEE Transactions on Education*, vol. 59, no. 3, pp. 202–209, Aug. 2016. DOI: 10.1109/TE.2015.2506624.

[13] T. McKenzie, M. Morales-Trujillo, S. Lukosch, and S. Hoermann, "Is Agile Not Agile Enough? A Study on How Agile is Applied and Misapplied in the Video Game Development Industry," *Proceedings - 2021 IEEE/ACM Joint 15th International Conference on Software and System Processes and 16th ACM/IEEE International Conference on Global Software Engineering, ICSSP/ICGSE 2021*, pp. 94–105, May 2021. DOI: 10.1109/ICSSP-ICGSE52873.2021.00019.

[14] W. Van der Aalst, "Process mining: Data science in action," *Process Mining: Data Science in Action*, pp. 1–467, Jan. 2016. DOI: 10.1007/978-3-662-49851-4.

[15] H. Ma, "Process-aware information systems: Bridging people and software through process technology," *JASIST*, vol. 58, pp. 455–456, Apr. 2007. DOI: 10.1002/asi.20456.

[16] W. Poncin, A. Serebrenik, and M. Brand, "Process Mining Software Repositories," in *Proceedings of the European Conference on Software Maintenance and Reengineering, CSMR*, Apr. 2011, pp. 5–14. DOI: 10.1109/CSMR.2011.5.

[17] S. Bala, A. Solti, C. Cabanillas, J. Mendling, and A. Polleres, "Mining Project-Oriented Business Processes," vol. 9253, Apr. 2015, ISBN: 978-3-319-23062-7. DOI: 10.1007/978-3-319-23063-4{_}28.

[18] V. Rubin, C. Günther, W. Aalst, E. Kindler, B. Dongen, and W. Schäfer, "Process Mining Framework for Software Processes," Apr. 2007, pp. 169–181, ISBN: 978-3-540-72425-4. DOI: 10.1007/978-3-540-72426-1{_}15.

[19] M. Arias, M. R. Marques, and E. Rojas, "Using process mining in agile software development methodologies: A systematic mapping study," *Proceedings - 2018 44th Latin American Computing Conference, CLEI 2018*, pp. 552–561, Oct. 2018. DOI: 10.1109/CLEI.2018.00072.

[20] A. M. Alsalemi and E. T. Yeoh, "A survey on product backlog change management and requirement traceability in agile (Scrum)," *2015 9th Malaysian Software Engineering Conference, MySEC 2015*, pp. 189–194, May 2016. DOI: 10.1109/MYSEC.2015.7475219.

[21] Z. Shehzadi, F. Azam, M. W. Anwar, and I. Qasim, "A novel framework for change requirement management (CRM) in agile software development (ASD)," *ACM International Conference Proceeding Series*, pp. 22–26, Aug. 2019. DOI: 10.1145/3357419.3357438.

[22] S. Qureshi, S. U. R. Khan, Inayat-Ur-Rehman, Y. Javed, S. Saleem, and A. Iqbal, "A Conceptual Model to Address the Communication and Coordination Challenges during Requirements Change Management in Global Software Development," *IEEE Access*, vol. 9, pp. 102 290–102 303, 2021, ISSN: 21693536. DOI: 10.1109/ACCESS.2021.3091603.

[23] P. Marnada, T. Raharjo, B. Hardian, and A. Prasetyo, "Agile project management challenge in handling scope and change: A systematic literature review," *Procedia Computer Science*, vol. 197, pp. 290–300, Jan. 2022, ISSN: 1877-0509. DOI: 10.1016/J.PROCS.2021.12.143.

[24] I. Mahmud and V. Veneziano, "Mind-mapping: An effective technique to facilitate requirements engineering in agile software development," *14th International Conference on Computer and Information Technology, ICCIT 2011*, pp. 157–162, 2011. DOI: 10.1109/ICCITECHN.2011.6164775.

[25] *Success Rates Rise — Pulse of the Profession 2017*. [Online]. Available: https://www.pmi.org/learning/thought-leadership/pulse/pulse-of-the-profession-2017.

[26] H. Doloi, "Cost Overruns and Failure in Project Management: Understanding the Roles of Key Stakeholders in Construction Projects," *Journal of Construction Engineering and Management*, vol. 139, no. 3, pp. 267–279, Mar. 2013, ISSN: 0733-9364. DOI: 10.1061/(ASCE)CO.1943-7862.0000621.

[27] P. Sharma and J. Singh, "Systematic literature review on software effort estimation using machine learning approaches," *Proceedings - 2017 International Conference on Next Generation Computing and Information Systems, ICNGCIS 2017*, pp. 54–57, Nov. 2018. DOI: 10.1109/ICNGCIS.2017.33.

[28] V. Yurdakurban and N. Erdoğan, "Comparison of machine learning methods for software project effort estimation," *26th IEEE Signal Processing and Communications Applications Conference, SIU 2018*, pp. 1–4, Jul. 2018. DOI: 10.1109/SIU.2018.8404495.

[29] M. Usman, K. Petersen, J. Börstler, and P. Santos Neto, "Developing and using checklists to improve software effort estimation: A multi-case study," *Journal of Systems and Software*, vol. 146, pp. 286–309, Dec. 2018, ISSN: 01641212. DOI: 10.1016/J.JSS.2018.09.054. [Online]. Available: https://www.researchgate.net/publication/327818006_Developing_and_Using_Checklists_to_Improve_Software_Effort_Estimation_a_Multi-Case_Study.

[30] S. Grimstad and M. Jørgensen, "Inconsistency of expert judgment-based estimates of software development effort," *Journal of Systems and Software*, vol. 80, no. 11, pp. 1770–1777, Nov. 2007, ISSN: 0164-1212. DOI: 10.1016/J.JSS.2007.03.001.

[31] M. O. Elish, T. Helmy, and M. I. Hussain, "Empirical study of homogeneous and heterogeneous ensemble models for software development effort estimation," *Mathematical Problems in Engineering*, vol. 2013, 2013, ISSN: 1024123X. DOI: 10.1155/2013/312067. [Online]. Available: https://www.researchgate.net/publication/258392154_Empirical_Study_of_Homogeneous_and_Heterogeneous_Ensemble_Models_for_Software_Development_Effort_Estimation.

[32] E. Kocaguneli, T. Menzies, and J. W. Keung, "On the value of ensemble effort estimation," *IEEE Transactions on Software Engineering*, vol. 38, no. 6, pp. 1403–1416, 2012, ISSN: 00985589. DOI: 10.1109/TSE.2011.111.

[33] *Behavior-driven-development - Wikipedia*. [Online]. Available: https://en.wikipedia.org/wiki/Behavior-driven-development.

[34] *Time to Market: What it is & Why it's Important — TCGen*. [Online]. Available: https://www.tcgen.com/time-to-market/.

[35] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *NAACL HLT 2019 - 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies - Proceedings of the Conference*, vol. 1, pp. 4171–4186, Oct. 2018. DOI: 10.48550/arxiv.1810.04805. [Online]. Available: https://arxiv.org/abs/1810.04805v2.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:21:19 UTC from IEEE Xplore. Restrictions apply.